

KraTeus Engine

Visão, Arquitetura e Diretrizes Técnicas — Documento Atualizado

Escopo: este documento descreve a engine e suas tecnologias. O jogo RTS é tratado apenas como um caso de uso futuro e não define a arquitetura da engine.

1. Identidade da engine

KraTeus Engine será uma engine própria, modular, escalável e orientada a desempenho. O objetivo não é competir inicialmente como uma engine universal gigantesca, mas construir uma base tecnológica sólida capaz de sustentar diferentes tipos de jogos e simulações.

Princípios: desempenho proporcional ao que é usado; arquitetura modular; dados separados da lógica; processamento local; escalabilidade; ferramentas profissionais; baixa sobrecarga.

2. Linguagem e base

Rust é a candidata principal para o núcleo da KraTeus.

- desempenho próximo ao nativo
- controle explícito de memória sem garbage collector tradicional
- segurança de memória
- concorrência forte
- boa adequação a ECS, simulação e processamento paralelo
- ecossistema adequado para ferramentas de baixo nível

Lua pode existir posteriormente como linguagem embutida para scripting, regras, eventos e modding, sem substituir Rust no núcleo.

3. Arquitetura geral

```
KraTeus Engine
■■■ Core
■■■ ECS
■■■ Simulation
■■■ Physics
■■■ World
■■■ Navigation
■■■ Renderer
■■■ RHI
■■■ Audio
■■■ Networking
■■■ Assets / Pipeline
■■■ UI
■■■ Save / Replay
■■■ Debug / Profiler
■■■ Editor / Tools
```

4. Runtime separado do Editor

O **Runtime** será independente do editor. O editor é uma ferramenta de desenvolvimento e não deve ser obrigatório no executável final. Isso reduz consumo e permite criar builds menores.

Editor → cria/configura/inspeciona
Runtime → executa o projeto
Engine Core → fornece os sistemas comuns

5. Modelo de processamento

A engine utilizará arquitetura orientada a dados, ECS e processamento em etapas. Sistemas independentes poderão trabalhar em paralelo quando não houver dependências entre eles.

```
Input
↓
Simulation / ECS
↓
Physics / AI / Navigation
↓
Render World
↓
Renderer
↓
GPU
```

6. Física e simulação

A física será um subsistema reutilizável e modular. A engine poderá calcular movimento, colisões, forças, gravidade, trajetórias e outros fenômenos sem depender de animações pré-programadas.

- Rigid bodies
- Collision shapes
- Gravity e forças
- Impulsos
- Joints e constraints
- Raycasts e sweeps
- Triggers
- Character controller
- Projectiles e balística
- Simulação determinística quando necessária

Exemplo: uma ferramenta pode receber altura e distância de um salto e calcular velocidade inicial, trajetória, altura máxima, tempo de subida, tempo de descida e tempo total de voo. O sistema pode oferecer diferentes níveis de fidelidade para evitar custo desnecessário.

7. Renderização

O renderer não ficará preso a uma API gráfica específica. A arquitetura terá uma **RHI (Rendering Hardware Interface)** entre o renderer e as APIs de GPU.

```
Game / Editor
↓
Render World
↓
Renderer
↓
RHI
■■■ Vulkan
■■■ DirectX 12
■■■ Metal
```

A implementação inicial de APIs específicas será decidida após pesquisa e protótipos. A prioridade é não contaminar o restante da engine com dependências de uma única API.

8. GPU-driven rendering e escalabilidade

- GPU instancing
- Batching
- Frustum/visibility culling
- LOD
- Streaming
- Renderização por lotes
- Redução de draw calls
- Uso eficiente de buffers e texturas
- Separação entre dados de simulação e dados necessários ao renderer

A engine não deve renderizar tudo na máxima qualidade. Objetos distantes, invisíveis ou irrelevantes devem consumir muito menos recursos.

9. 2D, 3D e qualidade gráfica

A KraTeus deverá suportar desde pixel art e 2D tradicional até 3D avançado e resoluções como 4K. A engine não deve impor uma estética visual ao projeto.

Pixel Art → 2D → 2.5D → Stylized 3D → Realistic 3D → High-end 4K

O custo deve ser proporcional aos recursos utilizados. Qualidade de sombras, pós-processamento, partículas, materiais, resolução interna e outros recursos serão configuráveis.

10. Render World

O mundo lógico e o mundo de renderização serão separados. A simulação pode conter muitas entidades enquanto o renderer recebe apenas os dados necessários para desenhar o estado atual.

```
GAME WORLD
Entidades / Componentes / Simulação
↓ extração
RENDER WORLD
Meshes / Sprites / Lights / Cameras / Materials / Visibility
↓
GPU
```

11. Memória e leveza

- Sistemas opcionais e carregados apenas quando necessários
- Pools e arenas quando fizerem sentido
- Layouts de dados favoráveis a cache
- Streaming de assets
- LOD e culling
- Evitar alocações por frame
- Descarregamento de recursos não utilizados

- Separação entre editor e runtime

12. Editor da KraTeus

O editor terá identidade visual própria: moderno, técnico, limpo e preferencialmente dark-first. A interface deve priorizar produtividade em vez de efeitos visuais.

- Scene / World Editor
- Asset Browser
- Inspector
- Material Editor
- Animation Tools
- Physics Preview
- Profiler
- Console e Logs
- Debugging
- Build / Packaging

13. Pipeline de assets

Assets serão processados por uma pipeline própria, convertendo formatos de origem para formatos otimizados para runtime. O objetivo é que o jogo não precise carregar diretamente os arquivos de autoria.

Source Asset → Importer → Processor → Optimized Asset → Runtime

14. Mundo e streaming

- World dividido em chunks
- Carregamento/descarregamento dinâmico
- Suporte a mundos grandes
- Streaming independente de renderização
- Possibilidade de múltiplas camadas de navegação

15. Networking, save e replay

Mesmo que nem todo projeto use multiplayer, a arquitetura deve evitar decisões que tornem networking e determinismo impossíveis posteriormente. Save/load e replay também devem ser considerados desde cedo na simulação.

16. Profiling e testes

- CPU profiler
- GPU profiler
- Memory profiler
- Frame timings
- Entity/system timings

- Stress tests
- Benchmarks automatizados
- Testes de determinismo
- Logs e diagnóstico

17. Processamento local

Em princípio, o jogo executará a simulação, física, IA e renderização na máquina do usuário. A engine deve aproveitar CPU e GPU locais e adaptar sua carga às capacidades disponíveis. Serviços externos só serão necessários quando um projeto optar por recursos como servidores multiplayer, serviços online ou telemetria.

18. Filosofia de performance

Regra central: o custo da engine deve acompanhar o que o projeto realmente utiliza.

Projeto	Objetivo
2D / Pixel Art	Baixa sobrecarga e pipeline simples
3D leve	Renderer e física escaláveis
Simulação grande	CPU/ECS/Jobs otimizados
3D avançado / 4K	Aproveitar GPU e recursos gráficos disponíveis

19. O que não fazer

- Não criar uma engine monolítica onde tudo depende de tudo.
- Não obrigar jogos simples a carregar sistemas avançados.
- Não prender o renderer a uma única API gráfica.
- Não misturar lógica de jogo com código de renderização.
- Não otimizar apenas por opinião: medir com benchmarks.
- Não começar por centenas de features antes de validar o núcleo.

20. Roadmap técnico inicial

Fase	Objetivo
Fase 1	Workspace Rust, Core, logging, configuração, testes e convenções
Fase 2	ECS, memória, scheduler/jobs e loop de simulação
Fase 3	RHI e primeiro backend gráfico
Fase 4	Renderer 2D/3D básico + recursos GPU
Fase 5	Physics + ferramentas de visualização
Fase 6	World, assets, streaming e editor inicial
Fase 7	Profiler, stress tests, save/replay e robustez
Fase 8	Backends adicionais, networking e recursos avançados

21. Estrutura inicial do repositório

```
krateus/  
■■■ Cargo.toml  
■■■ crates/  
■ ■■■ krateus-core/  
■ ■■■ krateus-ecs/  
■ ■■■ krateus-simulation/  
■ ■■■ krateus-physics/  
■ ■■■ krateus-render/  
■ ■■■ krateus-rhi/  
■ ■■■ krateus-world/  
■ ■■■ krateus-navigation/  
■ ■■■ krateus-audio/  
■ ■■■ krateus-networking/  
■ ■■■ krateus-editor/  
■■■ assets/  
■■■ examples/  
■■■ tests/  
■■■ tools/  
■■■ docs/
```

22. Decisões ainda abertas

- Qual backend gráfico será o primeiro.
- Qual toolkit de UI será usado no editor.
- Qual biblioteca de física será integrada ou se algum subsistema será próprio.
- Estratégia exata de ECS e scheduler.
- Formato binário dos assets.
- Sistema de shaders e material graph.
- Plataformas-alvo iniciais.

23. Princípio final

KraTeus Engine deve ser uma plataforma tecnológica em que desempenho, memória, renderização, física e ferramentas são sistemas independentes, mensuráveis e escaláveis. A engine deve ser capaz de crescer sem obrigar todos os projetos a pagar pelo máximo de recursos.